Head First Software Development: A Learner's Companion to Software Development - Dan Pilone, Russ Miles - Google Knjige

Problem 1

Write a short paragraph to answer these three questions:

- What are the two major concerns of any software project?
- Which of those two do you feel is more important?
- Where does the idea of complete functionality fit with these two concerns?

There are two major concerns to any software project: “how much will it cost?” and “how long will it take?”. In my opinion, whichever is more important depends on the customer’s standing, but generally “how much will it cost” would trump the importance of “how long will it take?”. Generally, time costs money, so disregarding deadlines for the project would likely require a higher cost. Higher costs are more likely to be a hard cap for a customer, as companies may be limited in how much they can spend on the project. The end goal of balancing these is to have a project with complete functionality! After all the dust, money, and time has settled, the final result should be the project fully functional, and if it needs to be back in production to get there, time and money still need to be spent.

Problem 2

Write a short paragraph to answer these three questions and briefly explain your opinion:

In the Agile method for software development, what are the five main phases that occur in each and every iteration?
Do you feel that any of them could be done at the start of the project and not be repeated in every iteration?
Do you feel that would save time overall on the project?

When developing with Agile, each iteration applies the following steps: ideation, development, testing, deployment, and operations. While each is helpful to consider for each iteration, frankly on smaller projects some can be disregarded by being planned ahead. For example, a simple calendar widget on a webpage might be developed with Agile, but the idea of how to implement it may not be needed to reconsider. What I see

as crucial to every iteration is development, testing, and deployment. It would save time in many cases of small works, but ignoring ideation and operations for larger projects can be detrimental and set a team up for getting stuck without a plan out.

Problem 3

The main phases that occur during Waterfall method for software development are usually requirements, analysis, design, implementation, testing, and deployment which are usually followed by maintenance. To differentiate from Agile, Waterfall completes each phase before moving to the next. Some Waterfall models will also recognize a separate feasibility study before development and a maintenance phase after delivery. I think feasibility assessment can be useful because it helps with determining whether a project is practical. An Agile team might still need a feasibility study, which is a Waterfall phase, before attempting to integrate its application with an unfamiliar external system.

Problem 4

1. User story is a short description, written from the customer's perspective, of something the software needs to do.
2. Blueskying is brainstorming possible features with the customer while encouraging ideas without immediately judging their practicality.
3. User stories should describe a customer need, focus on one feature, use language the customer understands, and remain short and clear.
4. User stories should not specify technical implementation details, combine several unrelated features, or rely on unconfirmed assumptions about what the customer wants.
5. Traditional Waterfall usually documents requirements in detailed requirements specification rather than user stories, although a team can use stories to help gather those requirements.

Problem 5

What is your opinion on the following statements, and why do you feel that way:

“All assumptions are bad, and no assumption is a good assumption.”
I don’t agree with this statement, as not all assumptions are bad; some assumptions are necessary to progress with the project when your information is lacking.

“A big user story estimate is a bad user story estimate.”
I agree with this statement because a big user story estimate means the story is too complex and needs to be shortened; a small estimate means the story is easier to understand and estimate.

Problem 6

Fill in the blanks in the statements below, using the following things [you can use each thing for more than one statement]: Blueskying; Role playing; Observation; User story; Estimate; Planning poker.

You can dress me up as a use case for a formal occasion: _____**User Story**_____
The more of me there are, the clearer things become: _____**User Story**_____
I help you capture EVERYTHING:
_____**Blueskying,Observation**_____
I help you get more from the customer: _____**Role Playing, Observation**_____
In court, I'd be admissible as firsthand evidence:
_____**Observation**_____
Some people say I'm arrogant, but really I'm just about confidence:
_____**Estimate**_____
Everyone's involved when it comes to me:
_____**Blueskying**_____

NOTE: when you have finished, check your answers with the result in your text on page 62. Do you agree with the book answers? If you disagree with any of them, justify your preferred answer.
Yeah, I agree with every answer in the textbook.

Problem 7

Explain what is meant by a better than best-case estimate.

A better than best-case estimate (referred to by Pilone & Miles as estimating in "Utopian days") is an estimate that assumes not only zero interruptions, bugs, or roadblocks, but also relies on unverified, overly optimistic hidden assumptions that reduce the actual scope of the work required. It represents a timeline that cannot be achieved even under ideal real-world conditions because it omits both operational overhead and necessary technical implementation steps.

Problem 8

In your opinion, when would be the best time to tell your customer that you will NOT be able to meet her delivery schedule? Why do you feel that is the best time? Do you think that would be a difficult conversation? If so, how could you make it less difficult?

When is the best time to tell the customer? Immediately, the exact moment your sanity-check, velocity calculation, or burndown rate reveals that the features will not fit into the schedule.

Wait until just before a deadline to tell a customer, you leave them the shortest amount of time possible to plan their response, leaving them little way to plan a reaction. Communicating the delay immediately preserves the maximum amount of time and opportunity for your customer to replan.
It would be a difficult conversation. Customers want their software as soon as possible, and often have obligations to higher ups and release dates and business planning that rely upon software coming at a certain time.

When delivering bad news, it is best to use concrete artifacts, specifically your Planning Poker user story estimates, your team's historical velocity, and your Big Project Board / burn-down chart. Keep the customer involved at every iteration boundary and give her visibility into the project board and burn-down chart.

Problem 9

Write a short paragraph to discuss why you think branching in your software configuration is bad or good, then describe a scenario to support your opinion.

I think branching in our software configuration and in general is a good thing. Branching, when used and managed correctly, allows us to work on things in the project without messing with each other's work or the main parts. However, having many branches at the same time can make things complicated especially when trying to merge changes back together from different people at different times. For example, if a group is trying to develop a web app and someone is working on a feature while someone else is fixing bugs, branching would let both developers work separately without risking each other's work.

Problem 10

Have you used a build tool in your development? If you have, which tool have you used? What are its good points and bad points — in other words, what do you like about it and/or dislike about it?

I have used npm or Node Package Manager while working on previous JavaScript and React projects. It is also likely to be used in this one with our planned React/nodejs structure. One thing that I like about npm is that it makes the managing dependencies and running build scripts much easier, since you can install the packages you need and build the project with a few commands. It helps keep the development process consistent when multiple people are working on the same project. One downside is that dependency conflicts or version issues can sometimes cause errors that are annoying to troubleshoot especially with multiple people. Overall, I think it saves time and makes building and managing projects more convenient.